Table 1.2 Time Complexity of Binary

Cases	Suppose Element (say x) present in Array A	Element x, not present in A
Best:	1 step required $\Rightarrow O(1)$	$O(\log n)$
Worst:	$\log n$ steps required $\Rightarrow O(\log n)$	$O(\log n)$
Average	$\log n$ steps required $\Rightarrow O(\log n)$	$O(\log n)$

1.7 ASYMPTOTIC NOTATIONS (O, Ω, and θ)

Asymptotic notations have been used in earlier sections in brief. In this section we will elaborate these notations in detail. They will be further taken up in the next unit of this block.

These notations are used to describe the Running time of an algorithm, in terms of functions, whose domains are the set of natural numbers, $N = \{1, 2, \ldots\ldots\}$. Such notations are convenient for describing the worst case running time function. T(n) (problem size input size).

The complexity function can be also be used to compare two algorithms P and Q that perform the same task.

The basic Asymptotic Notations are:

1. O(Big-"Oh") Notation. [Maximum number of steps to solve a problem, (upper bound)]
2. Ω (Big-"Omega") Notation [Minimum number of steps to solve a problem, (lower bound)]
3. Θ(Theta) Notation [Average number of steps to solve a problem, (used to express both upper and lower bound of a given $f(n)$]

1.7.1 The Notation O (Big 'Oh')

Big 'Oh' Notation is used to express an asymptotic upper bound (maximum steps) for a given function $f(n)$. Let $f(n)$ and $g(n)$ are two positive functions , each from the set of natural numbers (domain) to the positive real numbers.

We say that the function $f(n) = O(g(n))$ [read as "f of n is big "Oh" of g of n"], if there exist two positive constants C and n_0 such that

$f(n) \leq C.g(n)$: $\forall$ $n \geq n_0$

The intuition behind O- notation is shown in Figure 1.1

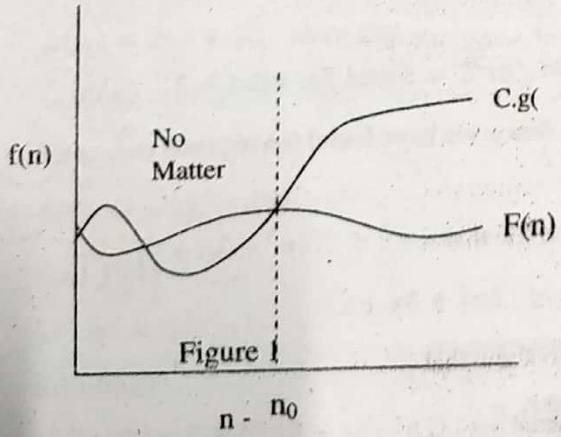

Figure 1.1: O-notation

For all values of n to the right of n_0, the value of f(n) is always lies on or below $Cg(n)$.

To understand O-Notation let us consider the following examples:

Example1\1: For the function defined by $f(n) = 2n^2 + 3n + 1$: show that

$(i) f(n) = O(n^2)$

$(ii) f(n) = O(n^3)$

$(iii) n^2 = O(f(n))$

$(iv) f(n) \neq O(n)$

$(v) n^3 \neq O(f(n))$

Solution:

(i) To show that $f(n) = O(n^2)$; we have to show that

$f(n) \leq C.g(n) \ \forall n \geq n_0$

$\Rightarrow 2n^2 + 3n + 1 \leq C.n^2$ ..... (1)

clearly this equation (1) is satisfied for $C = 6$ and for all $n \geq 1$

$2n^2 + 3n + 1 \leq 6.n^2$ *for all* $n \geq 1$; since we have found the required constant C and n_0. Hence $f(n) = O(n^2)$.

Remark: The value of C and n_0 is not unique. For example, to satisfy the above equation (1), we can also take $c = 3$ *and* $n \geq 3$. So depending on the value of C, the value of n_0 also changes. Thus any value of C and n_0 which satisfy the given inequality is a valid solution.

(ii) To show that $f(n) = O(n^3)$; we have to show that

$f(n) \leq C.g(n) \ \forall n \geq n_0$

$\Rightarrow 2n^2 + 3n + 1 \leq C.n^3$ ..... (1) ; **Let C=3**

Value of n	$2n^2 + 3n + 1$	$3n^3$
$n = 1$	6	3
$n = 2$	15	24
$n = 3$	27	81
.....	.....	.....

clearly this equation (1) is satisfied for $C = 3$ and for all $n \geq 2$

$2n^2 + 3n + 1 \leq 3.n^2$ *for all* $n \geq 2$; Since we have found the required constant C and $n_0 = 2$. Hence $f(n) = O(n^3)$.

(iii) To show $n^2 = O(f(n))$ we have to show that $n^2 \leq C(2n^2 + 3n + 1)$

For $C = 1$ *and* $n \geq 1$; we get $n^2 \leq (2n^2 + 3n + 1)$.

(iv) To show $f(n) \neq O(n)$, you have to show that

$f(n) \leq C.n \Rightarrow 2n^2 + 3n + 1 \leq C.n$

or $(2n + 3 + \frac{1}{n}) \leq C$ ..... (1)

here is no value of C and n_0, which satisfy this equation (1). For example, if you take $C = 10^6$, then to contradict this inequality you can take any value greater than C, that is $n_0 = 10^7$. Since we do not found the required constant C and n_0 to satisfy (1). Hence $f(n) \neq O(n)$

(v) Do yourself.

Theorem: *If* $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots\cdots\cdots + a_1 n^1 + a_0$;

then $f(n) = O(n^m)$

1.7.2 The Notation Ω (Big 'Omega')

O- Notation provides an asymptotic upper bound for a function; Ω-Notation, provides an asymptotic *lower-bound* for a given function.

We say that the function $f(n) = \Omega(g(n))$ [read as "f of n is big "Omega" of g of n"], if and only if there exist two positive constants C and n_0 such that

$f(n) \geq C.g(n) : \forall\ n \geq n_0$

The intuition behind Ω- notation is shown in Figure 1.2

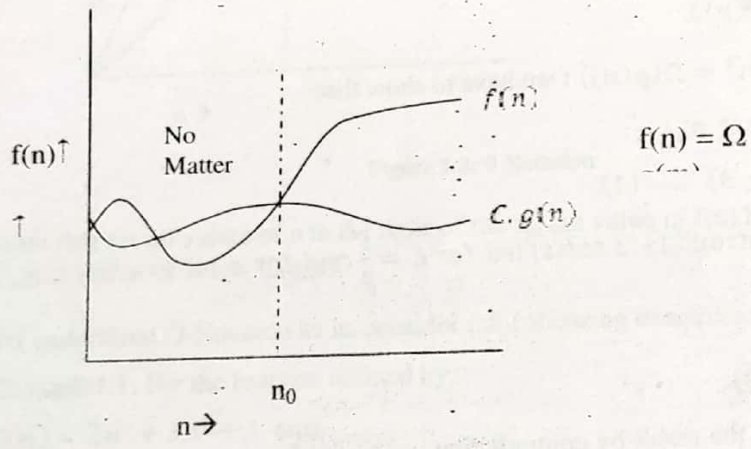

Figure 1.2: Ω notation

Note that in *figure 2* all values of f(n) always lies on or above g(n).

To understand Ω-Notation let us consider the following examples:

Example1.1: For the function defined by

$f(n) = 2n^3 + 3n^2 + 1$ *and*

$g(n) = 2n^2 + 3$: show that

(i) $f(n) = \Omega(g(n))$

(ii) $g(n) \neq \Omega(f(n))$

(iii) $n^3 = \Omega(g(n))$

(iv) $f(n) \neq \Omega(n^4)$

(v) $n^2 \neq \Omega(f(n))$

Solution:

(i) To show that $f(n) = \Omega(g(n))$; we have to show that

$f(n) \geq C.g(n)\ \forall n \geq n_0$

$\Rightarrow 2n^3 + 3n^2 + 1 \geq C(2n^2 + 3)$ .....(1)

clearly this equation (1) is satisfied for $C = 1$ *and for all* $n \geq 1$

Since we have found the required constant C and $n_0 = 1$.

Hence $f(n) = \Omega(g(n))$.

(ii) To show that $g(n) \neq \Omega(f(n))$; we have to show that no value of C and n_0 is there which satisfy the following equation (1). We can prove this result by contradiction.

Let $g(n) = \Omega(f(n)) \Rightarrow 2n^2 + 3 = \Omega(2n^3 + 3n^2 + 1)$ then there exist a positive constant C and n_0 such that

$2n^2 + 3 \geq C(2n^3 + 3n^2 + 1)\ \forall n \geq n_0$ ..... (1)

$(2 - 3C)n^2 \geq 2Cn^3 - 2 \geq Cn^3$

$\Rightarrow \frac{(2-3C)}{C} \geq n$ *for all* $n \geq n_0$

But for any value $> \frac{(2-3C)}{C}$; the inequality (1) is not satisfied $\Rightarrow$ Contradiction.

Thus $g(n) \neq \Omega(f(n))$.

(iii) To show that $n^3 = \Omega(g(n))$; we have to show that

$n^3 \geq C.g(n)\ \forall n \geq n_0$

$\Rightarrow n^3 \geq C(2n^2 + 3)$ .....(1)

clearly this equation (1) is satisfied for $C = \frac{1}{2}$ *and for all* $n \geq 2$ *(i.e.* $n_0 = 2$*)*

Thus $n^3 = \Omega(g(n))$

(iv) We can prove the result by contradiction.

Let $f(n) = \Omega(n^4) \Rightarrow 2n^3 + 3n^2 + 1 \geq C.n^4$ .....(1)

$\Rightarrow 6n^3 \geq C.n^4$ *for all* $n \geq n_0 \geq 1$

$\Rightarrow 6 \geq C.n \Rightarrow \frac{6}{C} \geq n$ *for all* $n \geq n_0$ .....(2)

But for $x = (\frac{6}{C} + 1)$ the inequality (1) or (2) is not satisfied

$\Rightarrow$ Contradiction, hence proved.

(v) Do yourself.

1.7.3 The Notation θ (Theta)

Θ-Notation provides simultaneous both asymptotic lower bound and asymptotic upper bound for a given function.

Let $f(n)$ and $g(n)$ are two positive functions , each from the set of natural numbers (domain) to the positive real numbers. In some cases, we have

f(n) = O(g(n) and f(n) = Ω(g(n)) then f(n) = θ(g(n)).

We say that the function $f(n) = \theta(g(n))$ [read as "f of n is Theta" of g of n"], if and only if there exist three positive constants C_1, C_2 and n_0 such that

$$C_1.g(n) \leq f(n) \leq C_2.g(n) \quad for\ all\ n \geq n_0 \qquad \ldots\ldots\ldots(1)$$

$$\leq f(n) \leq C_2.g(n);$$

(Note that this inequality (1) represents two conditions to be satisfied simultaneously viz $C_1.g(n) \leq f(n)$ and $f(n) \leq C_2.g(n)$;

clearly this implies

if $f(n) = O(g(n))$ *and* $f = \Omega(g(n))$ *then* $f(n) = \theta(g(n))$.

The 1.3 shows the intuition behind the Θ-Notation.

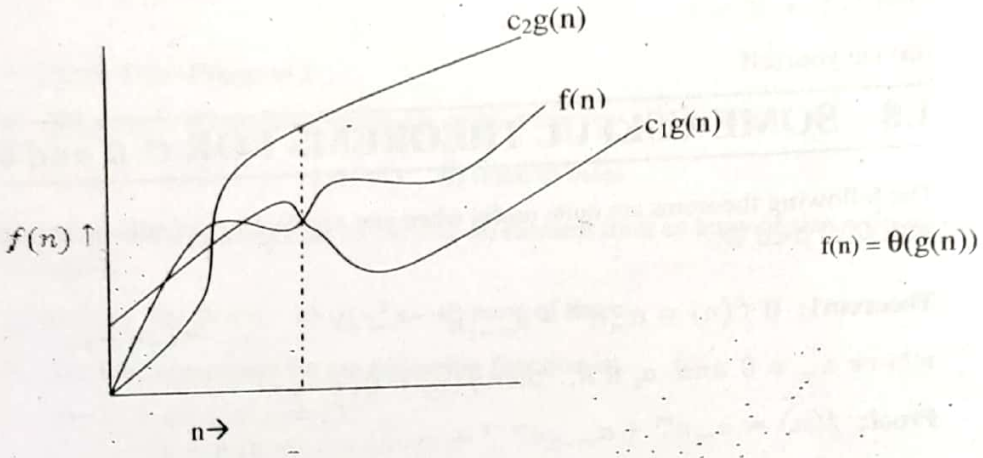

Figure 1.3: θ Notation

Note that for all values of n to the right of the n_0 the value of f(n) lies at or above $C_1g(n)$ and at or below $C_2.g(n)$.

To understand Ω-Notation let us consider the following examples:

Example1.1: For the function defined by

$f(n) = 2n^3 + 3n^2 + 1$ *and*

$g(n) = 2n^3 + 1$: show that

$(i) f(n) = \theta(g(n))$

$(ii)\ f(n) \neq \theta(n^2)$

$(iii)\ n^4 \neq \theta(g(n))$

Solution: To show that $(n) = \theta(g(n))$; we have to show that

$$C_1.g(n) \leq f(n) \leq C_2.g(n) \quad for\ all\ n \geq n_0$$

$$\Rightarrow C_1(2n^3 + 1) \leq 2n^3 + 3n^2 + 1 \leq C_2(2n^3 + 1)\ for\ all\ n \geq n_0 \quad \ldots.(1)$$

To satisfy this inequality (1) simultaneously, we have to find the value of C_1, C_2 and n_0 , using the following inequality

$$C_1(2n^3 + 1) \leq 2n^3 + 3n^2 + 1 \quad \ldots\ldots..(2) \quad and$$

$$2n^3 + 3n^2 + 1 \leq C_2(2n^3 + 1) \quad \ldots\ldots\ldots(3)$$

Inequality (2) is satisfied for $C_1 = 1$ *and* $n \geq 1$ $(i.e.\ n_0 = 1)$

Inequality (3) is satisfied for $C_2 = 2$ *and* $n \geq 1$ $(i.e.\ n_0 = 1)$

Hence inequality (1) simultaneously satisfied for $C_1 = 1, C_2 = 2$ and $n \geq$

Hence $f(n) = \Theta(g(n))$.

(ii) We can prove this by contradiction that no value of C_1, C_2 or n_0 exist.

Let $f(n) = \Theta(n^2) \Rightarrow C_1.n^2 \leq 2n^3 + 3n^2 + 1 \leq C_2.n^2$ for all $n \geq n_0$ ...(1)

Left side inequality: $C_1.n^2 \leq 2n^3 + 3n^2 + 1$; is satisfied for $C_1 = 1$ and $n \geq 1$.

Right side inequality: $2n^3 + 3n^2 + 1 \leq C_2.n^2$

$\Rightarrow n^3 \leq C_2.n^2 \Rightarrow n \leq C_2$ for all $n \geq n_0$;

But for $n = (C_2 + 1)$, this inequality is not satisfied.

Thus $f(n) = \Theta(n^2)$.

(iii) Do yourself.

1.8 SOME USEFUL THEOREMS FOR O, Ω and Θ.

The following theorems are quite useful when you are dealing (or solving problems) with O, Ω and Θ.

Theorem1: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots \cdots \cdots + a_1 n + a_0$

where $a_m \neq 0$ and $a_i \in R$, then $f(n) = O(n^m)$

Proof: $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots \cdots \cdots + a_1 n + a_0$

$= \sum_{i=0}^{m} a_k n^k$

$f(n) \leq \sum_{i=0}^{m} |a_k| n^k$

$\leq n^m \sum_{i=0}^{m} |a_k| n^{k-m} \leq n^m \sum_{i=0}^{m} |a_k|$ for $n \geq 1$

Let us assume $|a_m| + |a_{m-1}| + \cdots \cdots + |a_1| + |a_0| = c$

Then $f(n) \leq cn^m \Rightarrow f(n) = O(n^m)$.

Theorem2: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots \cdots \cdots + a_1 n + a_0$

where $a_m \neq 0$ and $a_i \in R$, then $f(n) = \Omega(n^m)$.

Proof: $f(n) = a_m n^m + \cdots \cdots + a_1 n + a_0$

Since $f(n) \geq cn^m$ for all $n \geq 1 \Rightarrow f(n) = \Omega(n^m)$

Theorem3: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots \cdots \cdots + a_1 n + a_0$

where $a_m \neq 0$ and $a_i \in R$, then $f(n) = O(n^m)$

Proof: From Theorem1 and Theorem2,

$f(n) = O(n^m)$ .....(1)

$f(n) = \Omega(n^m)$ ..... (2)

From (1) and (2) we can say that $f(n) = \Theta(n^m)$.

Example1: By applying theorem, find out the O-notation, Ω- notation and Θ-notation for the following functions.

(i) $f(n) = 5n^3 + 6n^2 + 1$

Hence inequality (1) simultaneously satisfied for $C_1 = 1, C_2 = 2$ and $n \geq$

Hence $f(n) = \Theta(g(n))$.

(ii) We can prove this by contradiction that no value of C_1, C_2 or n_0 exist.

Let $f(n) = \Theta(n^2) \Rightarrow C_1.n^2 \leq 2n^3 + 3n^2 + 1 \leq C_2.n^2$ for all $n \geq n_0$ ...(1)

Left side inequality: $C_1.n^2 \leq 2n^3 + 3n^2 + 1$; is satisfied for $C_1 = 1$ and $n \geq 1$.

Right side inequality: $2n^3 + 3n^2 + 1 \leq C_2.n^2$

$\Rightarrow n^3 \leq C_2.n^2 \Rightarrow n \leq C_2$ for all $n \geq n_0$;

But for $n = (C_2 + 1)$, this inequality is not satisfied.

Thus $f(n) = \Theta(n^2)$.

(iii) Do yourself.

1.8 SOME USEFUL THEOREMS FOR O, Ω and Θ.

The following theorems are quite useful when you are dealing (or solving problems) with O, Ω and Θ.

Theorem1: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots \cdots \cdots + a_1 n + a_0$

where $a_m \neq 0$ and $a_i \in R$, then $f(n) = O(n^m)$

Proof: $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots \cdots \cdots + a_1 n + a_0$

$= \sum_{i=0}^{m} a_k n^k$

$f(n) \leq \sum_{i=0}^{m} |a_k| n^k$

$\leq n^m \sum_{i=0}^{m} |a_k| n^{k-m} \leq n^m \sum_{i=0}^{m} |a_k|$ for $n \geq 1$

Let us assume $|a_m| + |a_{m-1}| + \cdots \cdots + |a_1| + |a_0| = c$

Then $f(n) \leq cn^m \Rightarrow f(n) = O(n^m)$.

Theorem2: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots \cdots \cdots + a_1 n + a_0$

where $a_m \neq 0$ and $a_i \in R$, then $f(n) = \Omega(n^m)$.

Proof: $f(n) = a_m n^m + \cdots \cdots + a_1 n + a_0$

Since $f(n) \geq cn^m$ for all $n \geq 1 \Rightarrow f(n) = \Omega(n^m)$

Theorem3: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots \cdots \cdots + a_1 n + a_0$

where $a_m \neq 0$ and $a_i \in R$, then $f(n) = O(n^m)$

Proof: From Theorem1 and Theorem2,

$f(n) = O(n^m)$ .....(1)

$f(n) = \Omega(n^m)$ ..... (2)

From (1) and (2) we can say that $f(n) = \Theta(n^m)$.

Example1: By applying theorem, find out the O-notation, Ω- notation and Θ-notation for the following functions.

(i) $f(n) = 5n^3 + 6n^2 + 1$

$f(n) = 7n^2 + 2n + 3$

Solution:

(i) Here *The degree of a polynomial* $f(n)$ *is,* $m = 3$, So by Theorem1,2 and 3:

$f(n) = O(n^3)$, $f(n) = \Omega(n^3)$ and $f(n) = \Theta(n^3)$.

(ii) *The degree of a polynomial* $f(n)$ *is,* $m = 2$, So by Theorem1,2 and 3:

$f(n) = O(n^2)$, $f(n) = \Omega(n^2)$ and $f(n) = \Theta(n^2)$.

Check Your Progress 1

1. $\sum_{i=1}^{n} O(n)$, Where O(n) stands for order n is:

   a) $O(n)$ b) $O(n^2)$ c) $O(n^3)$ d) none of these

2. What is the running time to retrieve an element from an array of size *n* (in worst case):

   a) $O(n)$ b) $O(n^2)$ c) $O(n^3)$ d) none of these

3. The time complexity for the following function is:

```
for(i = 1;i ≤ n;i += 2)
  { sum = 0;
  }
```

   a) $O(n)$ b) $O(n^2)$ c) $O(n\log n)$ d) $O(\log n)$

4. The time complexity for the following function is:

```
for(i = 1;i ≤ n;i++)
  for(j = 1;j ≤ n;j = j * 2)
  { ..... .....
      .....
  }
```

   a) $O(n)$ b) $O(n^2)$ c) $O(n\log n)$ d) $O((\log n)^2)$

5. Define an algorithm? What are the various properties of an Algorithm?

   .....

   .....

   .....

6. What are the various fundamental techniques used to design an algorithm efficiently? Also write two problems for each that follows these techniques?

   .....

   .....

   .....

7. Differentiate between profiling and debugging?

.....

.....

.....

8. Differentiate between asymptotic notations O, Ω and Θ?

.....

.....

.....

9. Define time complexity. Explain how time complexity of an algorithm is computed?

.....

.....

.....

10. Let f(n) and g(n) be two asymptotically positive functions. Prove or disprove the following (using the basic definition of O, Ω and Θ):

a) $4n^2 + 7n + 12 = O(n^2)$

b) $logn + \log(logn) = O(logn)$

c) $3n^2 + 7n - 5 = \Theta(n^2)$

d) $2^{n+1} = O(2^n)$

e) $2^{2n} = O(2^n)$

f) $f(n) = O(g(n))$ implies $g(n) = O(f(n))$

g) $\max\{f(n), g(n)\} = \Theta(f(n) + g(n))$

h) $f(n) = O(g(n))$ implies $2^{f(n)} = O(2^{g(n)})$

i) $f(n) + g(n) = \Theta(\min(f(n), g(n))$

j) $33n^3 + 4n^2 = \Omega(n^3)$

k) $f(n) + g(n) = O(n^2)$ where

$f(n) = 2n^2 - 3n + 5$ and $g(n) = nlogn + 10$

1.9 RECURRENCE

There are two important ways to categorize (or major) the effectiveness and efficiency of algorithm: ***Time complexity*** and ***space complexity***. The *time complexity* measures the amount of time used by the algorithm. The *space complexity* measures the amount of space used. We are generally interested to find the best case, average case and worst case complexities of a given algorithm. When a problem becomes "large", we are interested to find *asymptotic complexity* and O (Big-Oh) notation is used to quantify this.

Recurrence relations often arise in calculating the time and space complexity of algorithms. Any problem can be solved either by writing ***recursive*** algorithm or by writing ***non-recursive*** algorithm. A *recursive algorithm* is one which makes a

15. *Master method* provides a "cookbook" method for solving recurrences of the form: $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ where $a \geq 1$ and $b > 1$ are constants and $f(n)$ is an asymptotically positive function.

16. In master method you have to always compare the value of $f(n)$ with $n^{\log_b a}$ to decide which case is applicable. If $f(n)$ is asymptotically smaller than $n^{\log_b a}$, then case1 is applied. If $f(n)$ is asymptotically same as $n^{\log_b a}$, then case2 is applied. If $f(n)$ is asymptotically larger than $n^{\log_b a}$, and if $af\left(\frac{n}{b}\right) \leq c.f(n)$ for some $c < 1$, then case3 is applied.

Master method is sometimes fails (either case1 or case 3) to solve a recurrence $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, as discussed above.

1.11 ANSWERS TO CHECK YOUR PROGRESS

Check Your Progress 1

1-b, 2-a, 3-d, 4-c

5. **An Algorithm** is a well defined computational procedure that takes input and produces output. Or we can say that an Algorithm is a finite sequence of instructions or steps (i.e. inputs) to achieve some particular output. Any Algorithm must satisfy the following criteria (or Properties)
   1. **Input**: It generally requires finite no. of inputs.
   2. **Output:** It must produce at least one output.
   3. **Uniqueness**: Each instruction should be clear and unambiguous
   4. **Finiteness:** It must terminate offer a finite no. of steps.

6. There are basically 5 fundamental techniques are used to design an algorithm efficiently:

Algorithm design techniques	Examples
Divide and Conquer	Binary search, Merge sort
Greedy Method	Knapsack problem, Minimum cost spanning tree problem (Kruskal's and Prim's Algorithm)
Dynamic Programming	All Pair Shortest Path Problem (Floyed Algorithm), Chain Matrix multiplication.
Backtracking	N-Queen's problem, Sum-of-subset problem
Branch and Bound	Assignment problem, TSP (Travelling salesman problem)

7. Testing a program consists of two phases: debugging and profiling (or performance measurement). Debugging is the process of executing programs on sample data sets to find whether wrong results occur and, if so, to correct them. Debugging can only point to the presence of errors.

Profiling is the process of executing a correct program on data sets and measuring its time and space. These timing figures are used to confirm the previous analysis and point out logical errors. These are useful to judge a better algorithm.

8. There are 3 basic asymptotic notations (O,Ω and Θ) used to express the time complexity of an algorithm.

O (Big-Oh) notation	$O(g(n) = \{f(n)$: there exist a positive constants c and n_0 such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0\}$. It express upper bound of a function $f(n)$.
Ω (Big omega) notation	$\Omega(g(n) = \{f(n)$: there exist a positive constants c and n_0 such that $0 \leq f(n) \geq cg(n)$ for all $n \geq n_0\}$. It express lower bound of a function $f(n)$.
Θ (Theta) notation	$\Theta(g(n) = \{f(n)$: there exist a positive constants c_1, c_2, and and n_0 such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0\}$. It express both upper and lower bound of a function $f(n)$.

9. **Time complexity**: The number of machine instructions which a program executes during its running time is called its *time complexity*. This number depends primarily on the size of the program's input. Time taken by a program is the sum of the compile time and the run time. In time complexity, we consider run time only. The time required by an algorithm is determined by the number of elementary operations.

The following primitive operations that are independent from the programming language are used to calculate the running time:

- Assigning a value to a variable
- Calling a function
- Performing an arithmetic operation
- Comparing two variables
- Indexing into a array of following a pointer reference
- Returning from a function

The following fragment shows how to count the number of primitive operations executed by an algorithm.

```
int sum(int n)
{
int i, sum;
1. sum = 0;                          //add 1 to the time count
2. for(i = 0; i < n; i++)            //add n + 1 to the time count
   {
3.   sum = sum + i * i;              //add n to the time count
   }
4.  return sum;                      //add 1 to the time count
```

This function returns the sum from $i = 1$ to n of i squared, i.e. $sum = 1^2 + 2^2 + \cdots \ldots + n^2$.

To determine the running time of this program, we have to count the number of statements that are executed in this procedure. The code at line 1 executes 1 time, at line 2 the **for loop** executes $(n+1)$ time, Line 3 executes n times, and line 4 executes 1 time. Hence the sum is $= 1 + (n+1) + n + 1 = 2n + 3$.

In terms of O-notation this function is $O(n)$.

10. a) $(4n^2 + 7n + 12) \leq c.n^2 \ldots\ldots(1)$

for c = 5 and $n \leq 9$; the above inequality (1) is satisfied.

Hence $4n^2 + 7n + 12 = O(n^2)$.

b) By using basic definition of Big – "oh" Notation:

$\log n + \log(\log n) \leq C.\log n \quad \ldots\ldots(1)$

For C = 2 and $n_0 = 2$, we have

$\log_2 2 + \log(\log_2 2) \leq 2.\log_2 2$

$\Rightarrow \quad 1 + \log(1) \leq 2$

$\Rightarrow \quad 1 + 0 \leq 2$

$\Rightarrow$ satisfied for c = 2 and $n_0 = 2$

$\Rightarrow \log n + \log(\log n) = O(\log n)$

c) $3n^2 + 7n - 5 = O(n^2)$;

To show this, we have to show:

$C_1.n^2 \leq 3n^2 + 7n - 5 \leq C_1.n^2 \quad \ldots\ldots(*)$

(i) L.H.S inequality:

$C_1.n^2 \leq 3n^2 + 7n - 5 \quad \ldots(1)$

This is satisfied for $C_1 = 1$ and $n \geq 2$

(ii) R.H.S inequality:

$3n^2 + 7n - 5 \leq C_2.n^2 \ldots(2)$

This is satisfied for $C_1 = 1$ and $n \geq 1$

$\Rightarrow$ inequality (*) is simultaneously satisfied for

$C_1 = 1, C_2 = 10$ and $n \geq 2$

d) $2^{n+1} \leq C.2^n \Rightarrow 2^{n+1} \leq 2.2^n$

e) $2^{2n} \leq C.2^n$

$\Rightarrow \quad 4^n \leq 2.2^n \ldots\ldots(1)$

No value of C and n_0 Satisfied this in equality (1)

$\Rightarrow \quad 2^{2n} \neq O(2^n)$.

f) No; $f(n) = O(g(n))$ does not implies $g(n) = O(f(n))$

Clearly $n = O(n^2)$, but $n^2 \neq O(n)$

g) To prove this, we have to show that

$$C_1.(f(n) + g(n) \leq \max\{f(n), g(n)\} \leq C_2(f(n) + g(n) \ldots\ldots\ldots\ldots (*)$$

1) L.H.S inequality:

$C_1.(f(n) + g(n) \leq \max\{f(n), g(n)\} \ldots\ldots\ldots\ldots (1)$

Let $h(n) = \max\{f(n), g(n)\} = \begin{cases} f(n) & \text{if } f(n) > g(n) \\ g(n) & \text{if } g(n) > f(n) \end{cases}$

$\therefore C_1.(f(n) + g(n) \leq f(n) \ldots\ldots\ldots (1)$

[Assume $\max\{f(n), g(n)\} = f(n)$]

for $C_1 = \frac{1}{2}$ and $n \geq 1$, this inequality (1) is satisfied:

since $\frac{1}{2}(f(n) + g(n) \leq f(n)$

$\Rightarrow f(n) + g(n) \leq 2f(n)$

$\Rightarrow f(n) + g(n) \leq f(n) + f(n)$ [$\because f(n) > g(n)$]

$\Rightarrow$ satisfied for $C_1 = \frac{1}{2}$ and $n \geq 1$

2) R.H.S. inequality

$\max\{f(n), g(n)\} \leq C_1.(f(n) + g(n) \ldots\ldots (2)$

This inequality (2) is satisfied for $C_2 = 1$ and $n \geq 1$

$\Rightarrow$ inqualtiy (*) is simultaneously satisfied for

$C_1 = \frac{1}{2}, C_2 = 1$ and $n \geq 1$

Remark : Let $f(n) = n$ and $g(n) = n^2$;

then $\max\{n, n^2\} = \Theta(n + n^2)$

$\Rightarrow n^2 = \Theta(n^2)$; which is TRUE(by definition of Θ)

h) *NO;* $f(n) = O(g(n)$ does not implies $2^{f(n)} = O(2^{g(n)})$;

we can prove this by taking a counter Example;

Let $f(n) = 2n$ and $g(n) = n$, we have

$2^{2n} = O(2^n)$; which is not TRUE [since $2^{2n} = 4^n \neq O(2^n)$].

i) *No,* $f(n) + g(n) \neq \Theta(\min\{f(n), g(n)\})$ We can prove this by taking counter example.

Let $f(n) = 2n$ and $g(n) = n^2$, then $(n + n^2) \neq \Theta(n)$

j) $(33n^3 + 4n^2) \geq C.n^4$; There is no positive integer for C and n_0 which satisfy this inequality. Hence $(33n^3 + 4n^2) \neq C.n^4$.

k) $f(n) + g(n) = 3n^2 + n + 4 + n\log n + 5 = h(n)$

$By\ O-notation\ h(n) \leq cn^2;$

This is true for $c = 4$ and $n_0 \geq 4$

Check Your Progress 2

1. a) At every step the problem size reduces to half the size. When the power is an odd number, the additional multiplication is involved. To find a time complexity of this algorithm, let us consider the ***worst case,*** that is we assume that at every step additional multiplication is needed. Thus total number of operations T(n) will reduce to number of operations for n/2, that is T(n/2) with three additional arithmetic operations(In odd power case: 2 multiplication and one division). Now we can write:

$T(n) = 1 \ if\ n = 0\ or\ 1$

$T(n) = T\left(\frac{n}{2}\right) + 3 \ \ if\ n \geq 2$

Instead of writing exact number of operations needed by the algorithm, we can use some constants. The reason for writing this constant is that we are always interested to find "asymptotic complexity" instead of finding exact number of operations needed by algorithm, and also it would not affect our complexity also.

$$T(n) = \begin{cases} T(1) = a & if\ n = 0\ or\ n = 1 \quad (\textbf{base case}) \\ T\left(\frac{n}{2}\right) - b & if\ n \geq 2 \quad (\textbf{Recursive step}) \end{cases}$$

b)

$$T(n) = \begin{cases} a & if\ n = 0\ or\ n = 1 \quad (\textbf{base case}) \\ T(n-1) + T(n-2) + b & if\ n \geq 2 \quad (\textbf{Recursive step}) \end{cases}$$

2. a)

$$T(n) = n + 3T\left(\frac{n}{2}\right)$$

$$= n + 3\left\{\frac{n}{2} + 3T\left(\frac{n}{4}\right)\right\} = n + \frac{3n}{2} + 3^2 T\left(\frac{n}{4}\right)$$

$$= n + 3.\frac{n}{2} + 3^2.\frac{n}{4} + T\left(\frac{n}{8}\right)$$

(By substituting $T\left(\frac{n}{4}\right)$ *in equation* 3) ; In this way we get the final GP series as:

$$= \underbrace{\left(n + \frac{3n}{2} + \frac{3^2 n}{4} + \cdots \cdots \frac{3^{k-1} n}{2^{k-1}}\right)}_{k=\log_2 n \ terms\ (total)} + T\left(\frac{n}{2^k}\right)$$

$$= n + \frac{3}{2}n + \left(\frac{3}{2}\right)^2 n + \cdots \cdots + \left(\frac{3}{2}\right)^{\log_2 n - 1} n + T(1)$$

$$= n\left(1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2 + \cdots \cdots + \left(\frac{3}{2}\right)^{\log_2 n - 1}\right) + a$$

$$= n.\frac{\left[\left(\frac{3}{2}\right)^{\log_2 n} - 1\right]}{\left(\frac{3}{2} - 1\right)} \quad [\text{ By using GP series sum formula } S_n = \frac{a(r^n - 1)}{r - 1}$$